

Neural Networks for Data Science

Master's Degree in Data Science

Lecture 2: Mathematical preliminaries

2a. Tensors and linear maps

Lecturer: S. Scardapane



SAPIENZA
UNIVERSITÀ DI ROMA

For the purpose of this course, an **ndarray** (informally, a *tensor*) is an array of elements of the *same type*, organized along one or more axes.

Each axis has a *meaning* that we encode in the letters we use, for example:

$\mathbf{x} \sim (D)$ a vector of **features**

$\mathbf{X} \sim (B, D)$ a **batch** (set) of vectors

$\mathbf{X} \sim (T, D)$ a sequence of vectors (**tokens**)

$X \sim (B, T, D)$ a batch of sequences

$X \sim (C, H, W)$ a color image (**channels**, height, width)

$X \sim (B, C, H, W)$ a batch of color images

$X \sim (B, T, C, H, W)$ a batch of sequences of images (videos)

A tensor is described both by its **shape** and by the type of its entries (**dtype**). We will mostly use three families:

```
1 X = torch.randn(32, 20, 128) # floating point
2 token_ids = torch.randint(5000, (32, 20)) # integers
3 mask = token_ids != 0 # booleans
```

- **floating point**: features, images, embeddings, activations, and parameters;
- **integers**: token IDs, class labels, and indices;
- **booleans**: masks.

For example, text tokenization produces integer IDs with shape (B, T) ; an embedding layer maps them to floating-point vectors with shape (B, T, D) .

Tensors are the default data structure in any deep learning framework:

```
1 import torch
2 X = torch.randn((3, 64, 64)) # shape (C, H, W)
```

NumPy-like indexing is pervasive (with 0-based indexing):

```
1 X[0, 0, 0] # full indexing (a scalar)
2 X[0] # shape (H, W)
3 X[:, 0] # shape (C, W)
```

In math, we use subscripts for scalar entries and brackets for slices:

$[X]_{0,:}$ 2-axis tensor of shape (H, W)

For $X \sim (B, T, D)$, consider a concrete tensor with $B = 32$, $T = 20$, and $D = 128$:

```
import numpy as np
X = np.empty((32, 20, 128)) # (B, T, D)
print("X[0, 3, :] ->", X[0, 3, :].shape)
print("X[0, :, :] ->", X[0, :, :].shape)
print("X[:, 3, :] ->", X[:, 3, :].shape)
```

Run

```
X[0, 3, :] -> (128,)
X[0, :, :] -> (20, 128)
X[:, 3, :] -> (32, 128)
```

0-axis tensors are called **scalars** (a physics terminology). Most scalars in this course are real-valued.

1-axis tensors are the classical **vectors** from linear algebra; they are written in boldface and treated as column vectors (when we care about the distinction):

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_D \end{pmatrix}, \quad \mathbf{x}^\top = (x_1 \quad x_2 \quad \cdots \quad x_D).$$

Vectors are either **feature vectors** (tabular datasets) or **embeddings** (of, e.g., pixels or text tokens). The Euclidean norm and the standard inner product are:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_d x_d^2}, \quad \langle \mathbf{x}, \mathbf{y} \rangle = \sum_d x_d y_d = \mathbf{x}^\top \mathbf{y}.$$

2-axis tensors are **matrices**. A matrix can represent a grid, a sequence of elements, or a stack (a **batch**) of vectors:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_B^\top \end{bmatrix} \sim (B, D).$$

For $\mathbf{X} \sim (B, D)$ and $\mathbf{W} \sim (D, H)$, matrix multiplication is defined by:

$$[\mathbf{X}\mathbf{W}]_{b,h} = \langle [\mathbf{X}]_{b,:}, [\mathbf{W}]_{:,h} \rangle = \sum_{d=1}^D X_{b,d} W_{d,h}$$

HIGHLIGHT INDEX

b

d

h

Geometrically, a matrix represents a *linear map* between two vector spaces:

$$\mathbf{W}^\top(\alpha\mathbf{x} + \beta\mathbf{y}) = \alpha\mathbf{W}^\top\mathbf{x} + \beta\mathbf{W}^\top\mathbf{y}, \quad \mathbf{W} \sim (D, H).$$

When applied to a batch of elements $\mathbf{X} \sim (B, D)$, matrix multiplication yields a transformed batch $\mathbf{H} = \mathbf{X}\mathbf{W} \sim (B, H)$:

$$[\mathbf{H}]_{b,:} = \mathbf{x}_b^\top \mathbf{W}.$$

Therefore $\mathbf{X}\mathbf{W} \sim (B, H)$ transforms a batch of vectors in $\mathbb{R}^D$ (e.g., a sequence of embeddings) into a transformed batch of vectors in $\mathbb{R}^H$.

Importantly, any operation applied to a batch **should not introduce** any dependency between the elements of the batch.

For $\mathbf{W} \sim (D, H)$, a **rank- r approximation** can be factorized as:

$$\mathbf{W} \approx \mathbf{U}\mathbf{V}^\top, \quad \mathbf{U} \sim (D, r), \quad \mathbf{V} \sim (H, r), \quad r \ll \min(D, H).$$

For a row vector $\mathbf{x}^\top$:

$$\mathbf{x}^\top \mathbf{W} \approx \sum_{k=1}^r (\mathbf{U}_k^\top \mathbf{x}) \mathbf{V}_k^\top$$

HIGHLIGHT TERM k
 $\mathbf{U}_k^\top \mathbf{x}$
 $\mathbf{V}_k^\top$

All linear maps can be decomposed into a sum of rank-1 operations operating on 1D subspaces.

Why we care: $\mathbf{U}\mathbf{V}^\top$ has rank at most r and uses $r(D + H)$ parameters instead of DH . This idea underlies parameter-efficient methods such as **low-rank adaptation** (LoRA).

Matrix multiplication combines element-wise products and summation over a matched axis.

Many operations we care about have a similar form. For example, summing over *two axes* with 2D arrays is a **matrix inner product**:

$$h = \sum_i \sum_j X_{i,j} Y_{i,j}$$

Matrix multiplication with an additional batching axis is **batched matrix multiplication**:

$$X \sim (B, T, D), \quad \mathbf{W} \sim (D, H), \quad Y \sim (B, T, H)$$

$$Y_{b,t,h} = \sum_{d=1}^D X_{b,t,d} W_{d,h}$$

HIGHLIGHT INDEX

b**t****d****h**

Mathematically, the operations above are examples of *tensor contractions*: summing over matched indices and retaining all the other axes (although not all arrays we manipulate are tensors in the proper sense).

Some operations extend to tensors by applying them independently to every element (**element-wise operations**):

$$[\exp(X)]_{i,j} = \exp(X_{i,j})$$

$$[X \odot Y]_{i,j} = X_{i,j} Y_{i,j} \quad \text{Hadamard product}$$

In NumPy and PyTorch, a very common way to express all these operations abstractly is through “einsum” (Einstein summation):

```
1 Y = torch.einsum("btd,dh->bth", X, W)
```

Before an element-wise operation, align the two shapes from the right:

- 1 Right-align** the axes. Missing leading axes are treated as having size **1**.
- 2** Aligned sizes p and q are compatible if $p = q$, $p = 1$, or $q = 1$.
- 3** The output size is $\max(p, q)$. Any incompatible pair raises an error.

LEFT	RIGHT	RESULT
(B, T, D)	(D)	(B, T, D)
(B, T, D)	(T, 1)	(B, T, D)
(32, 20, 128)	(32, 128)	error
(3, 1)	(1, 3)	(3, 3)

Check compatible shapes directly:

```
import numpy as np
X = (2, 3, 4) # (B, T, D)
for Y in [(4,), (3, 1), (2, 4)]:
    try: print(Y, "->", np.broadcast_shapes(X, Y))
    except ValueError: print(Y, "-> error")
```

Run

```
(4,) -> (2, 3, 4)
(3, 1) -> (2, 3, 4)
(2, 4) -> error
```

For a neural-network bias, $(B, H) + (H) \rightarrow (B, H)$: the same bias is added to every row of the batch.

The operations introduced so far already define a fully connected layer:

$$\mathbf{H} = \psi(\mathbf{X}\mathbf{W} + \mathbf{b}), \quad \mathbf{X} \sim (B, D), \quad \mathbf{H} \sim (B, H), \quad \mathbf{b} \sim (H).$$

XW

mix features (linearly)

+ b

broadcast over the batch

ψ

apply element-wise a non-linear
transformation

The parameters are $\theta = \{\mathbf{W}, \mathbf{b}\}$. How do we choose them?